

## TOPIC 01: API (APPLICATION PROGRAMMING INTERFACE)

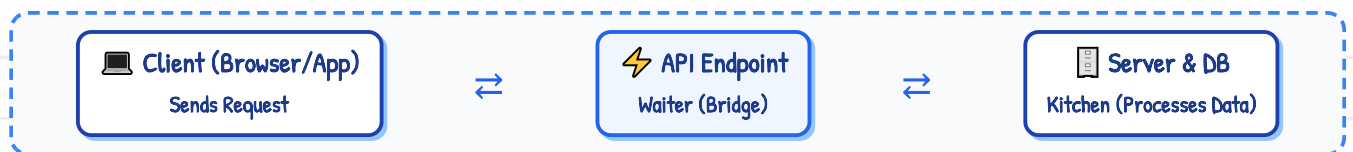
### What is an API? (Core Definition)

An API stands for **Application Programming Interface**. It acts as a **secure communication contract & bridge** between two independent software applications, allowing them to exchange data and services without needing to understand each other's underlying internal code or database structure.

### Real-World Analogy: The Restaurant Waiter

Think of dining at a restaurant:

- **Customer (Frontend / Client):** Sits at a table, looks at the menu, and places an order.
- **Kitchen (Backend / Database):** Prepares the actual food and holds all raw ingredients (database records).
- **Waiter (API):** Takes the customer's request to the kitchen, waits for the meal to be prepared, and safely delivers the final dish back to the customer's table!



### Industry API Use Cases:

- **Weather Services:** Mobile weather apps consume live forecast data from the **OpenWeather API**.
- **Payment Processing:** E-commerce stores use the **Stripe / Razorpay API** to charge cards securely.
- **OAuth Login:** Third-party sites authenticate users via **Google Identity API** without storing user passwords.



**Core Principle (Abstraction):** APIs deliver strong architectural decoupling. The client only needs to know the endpoint URI and payload format, keeping backend business logic completely abstracted.

## TOPIC 02: REST (REPRESENTATIONAL STATE TRANSFER)

### What is REST? (Definition & Architectural Origins)

REST stands for **Representational State Transfer**. It is **NOT** a protocol, **NOT** a library, and **NOT** a programming language!

It was introduced in the year 2000 by **Dr. Roy Fielding** in his doctoral dissertation. REST is an **Architectural Style**—a coordinated set of design principles and constraints that dictate how distributed hypermedia systems should communicate cleanly over HTTP.

### The Resource & Representation Model:

- **Resource:** Any key information entity managed by the server (e.g., users, posts, comments), addressed via a unique URI like `/posts/101`.
- **Representation:** When a client queries a resource, the server responds with a snapshot of its current state in a standardized format, overwhelmingly JSON (or XML).

### The 6 Guiding Constraints of REST Architecture:

- **1. Client-Server Separation:** Strict division of concerns; UI presentation and data persistence evolve independently.
- **2. Statelessness (The Golden Rule):** No client session context is retained on the server. Every incoming HTTP request must include complete authentication credentials and parameters.
- **3. Cacheability:** All server responses must explicitly label themselves as cacheable or non-cacheable to optimize network traffic.
- **4. Uniform Interface:** Consistent resource naming (nouns such as `/posts`) and standard HTTP methods (GET, POST, etc.).
- **5. Layered System:** Intermediary proxies, CDNs, and load balancers can sit transparently between client and server.
- **6. Code on Demand (Optional):** Servers may temporarily extend client functionality by transferring executable scripts (e.g., JS).

## TOPIC 03: API vs REST vs RESTFUL API (MASTER COMPARISON)

### The Definitive Architectural Comparison:

| Feature / Dimension | API                                                                       | REST                                                                 | RESTful API                                                             |
|---------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------|
| Definition          | A programming interface allowing two software components to interact.     | An architectural design style & set of constraints for web services. | A concrete web API implementation strictly adhering to REST principles. |
| Nature / Type       | Broad concept & communication mechanism.                                  | Design philosophy & architectural specification.                     | Tangible web service application / software product.                    |
| Protocols           | Any protocol (TCP, HTTP, IPC, library function calls).                    | Architectural pattern tailored for HTTP / web mechanics.             | Strictly operates over <b>HTTP / HTTPS</b> .                            |
| Data Formats        | Any format (JSON, XML, binary, plain text, in-memory objects).            | Format-agnostic (focuses on resource state).                         | Standardized primarily on <b>JSON</b> (or XML).                         |
| Constraint Rules    | No mandatory architectural rules imposed.                                 | Defines 6 fundamental architectural constraints.                     | Enforces all REST constraints directly in backend code.                 |
| Real Example        | <code>Math.random()</code> , <code>fs.readFile()</code> , Browser DOM API | Roy Fielding's 2000 Ph.D. dissertation specifications                | GitHub REST API, Twitter/X API, Express /posts CRUD API                 |

#### Golden Interview Rule:



"All RESTful APIs are APIs, but not all APIs are RESTful!"

(Just as all sports cars are vehicles, but not all vehicles are sports cars!)



**Crucial Interview Distinction:** Never say REST is a protocol or library! REST is an **Architectural Style** that operates on top of the HTTP protocol.

## TOPIC 04: HTTP METHODS & CRUD OPERATIONS MAPPING

### CRUD Operations & HTTP Verbs Matrix:

| CRUD Operation   | HTTP Verb | Example Endpoint  | Purpose & Behavior                            | Safe? | Idempotent? |
|------------------|-----------|-------------------|-----------------------------------------------|-------|-------------|
| Create           | POST      | POST /posts       | Submits new resource entity to the server.    | ✗ No  | ✗ No        |
| Read             | GET       | GET /posts        | Retrieves resource representations safely.    | ✓ Yes | ✓ Yes       |
| Update (Full)    | PUT       | PUT /posts/:id    | Replaces the entire targeted resource record. | ✗ No  | ✓ Yes       |
| Update (Partial) | PATCH     | PATCH /posts/:id  | Modifies only the specific fields supplied.   | ✗ No  | ✗ No*       |
| Delete           | DELETE    | DELETE /posts/:id | Permanently removes the specified resource.   | ✗ No  | ✓ Yes       |

### Crucial Distinction: PUT vs PATCH

- **PUT (Complete Replacement):** If a post contains { title, content, author } and you pass only { content: "New" }, the missing properties are overwritten and erased!
- **PATCH (Partial Modification):** Only touches the keys provided in the request body. Unmentioned properties remain completely intact!

#### Safety vs Idempotency:



- **Safe Method:** Read-only; causes zero state mutation on the server (e.g., GET).
- **Idempotent Method:** Calling the operation 1 time or 100 times leaves the server in the identical resulting state (e.g., GET, PUT, DELETE).

## TOPIC 05: STATUS CODES & TOPIC 06: RESTFUL ROUTES

### Standard HTTP Status Codes in REST APIs:

- **2xx (Success):** 200 OK (Standard success), 201 Created (Resource created via POST), 204 No Content (DELETE success).
- **3xx (Redirection):** 301 Moved Permanently, 302 Found / Temporary Redirect (Used in Express `res.redirect`).
- **4xx (Client Errors):** 400 Bad Request (Validation error), 401 Unauthorized (Missing token), 403 Forbidden (Denied permission), 404 Not Found (Resource missing).
- **5xx (Server Errors):** 500 Internal Server Error (Unhandled server exception).

### The 7 Canonical RESTful Routes for Posts Resource:

| Method | Route Path      | Action Name | Purpose & Functional Role                    | Express Response                     |
|--------|-----------------|-------------|----------------------------------------------|--------------------------------------|
| GET    | /posts          | Index       | List and display all posts                   | <code>res.render("index.ejs")</code> |
| GET    | /posts/new      | New         | Render HTML form to create a new post        | <code>res.render("new.ejs")</code>   |
| POST   | /posts          | Create      | Accept payload and save new post to database | <code>res.redirect("/posts")</code>  |
| GET    | /posts/:id      | Show        | Display details of a single post by ID       | <code>res.render("show.ejs")</code>  |
| GET    | /posts/:id/edit | Edit        | Render HTML form to edit an existing post    | <code>res.render("edit.ejs")</code>  |
| PATCH  | /posts/:id      | Update      | Update specific fields of an existing post   | <code>res.redirect("/posts")</code>  |
| DELETE | /posts/:id      | Destroy     | Remove targeted post from database           | <code>res.redirect("/posts")</code>  |

## TOPIC 07: RESPONSE HANDLING & TOPIC 08: UUID PACKAGE

### Express Response Handling: 3 Key Methods

- `res.send(data)`: Sends raw strings, HTML markup, or JSON objects. Standard choice when delivering JSON from REST APIs.
- `res.render("index.ejs", { posts })`: Compiles an EJS template with dynamic variables and sends the HTML document.
- `res.redirect("/posts")`: Issues an HTTP 302 redirect header instructing the browser to execute a fresh GET request.

#### Post-Redirect-Get (PRG) Pattern:



Never invoke `res.render()` directly after a POST, PATCH, or DELETE mutation! Always use `res.redirect('/posts')`. If the user hits refresh (F5), direct rendering prompts browser form resubmission, creating duplicate records!

### Topic 08: UUID Package (Universally Unique Identifiers)

#### Why Avoid Raw Array Indices (`posts[0]`)?

When an item is deleted (e.g. `posts.splice(0, 1)`), subsequent indices shift down. Any bookmarked link like `/posts/2` will suddenly display the incorrect resource!

#### The Industry Solution: UUID v4

- **UUID**: Universally Unique Identifier—a 128-bit cryptographically random unique string guaranteed never to collide.
- Installation: `npm install uuid`

```
const { v4: uuidv4 } = require('uuid');

// Generates a guaranteed unique ID string
const id = uuidv4();
console.log(id); // Output: "1b9d6bcd-bbfd-4b2d-9b5d-ab8dfbbd4bed"

// Used when instantiating new resources:
const newPost = { id: uuidv4(), username, content };
```

uuid-usage.js

## TOPIC 09: HTML FORMS & METHOD-OVERRIDE MIDDLEWARE

### HTML Form Specification Constraint:



Standard HTML `<form>` elements natively support ONLY two HTTP verbs: **GET** and **POST**! Specifying `method="PATCH"` or `method="DELETE"` causes browsers to fall back silently to a standard **GET** request!

### The Solution: method-override Middleware

This Express middleware inspects incoming requests for a designated query parameter (e.g., `?_method=PATCH`) and transparently mutates the internal HTTP method on the server.

### Installation & Express Configuration:

- Terminal Command: `npm install method-override`

```
app.js (Middleware Registration)

const methodOverride = require('method-override');

// Express overrides req.method based on query string
app.use(methodOverride('_method'));
```

### EJS Form Implementation in VS Code Style:

```
views/posts/edit.ejs & index.ejs

<!-- 1. EDIT FORM (Dispatches PATCH /posts/:id) -->
<form method="POST" action="/posts/<%= post.id %>?_method=PATCH">
  <textarea name="content"><%= post.content %></textarea>
  <button type="submit">Update Post</button>
</form>

<!-- 2. DELETE FORM (Dispatches DELETE /posts/:id) -->
<form method="POST" action="/posts/<%= post.id %>?_method=DELETE">
  <button type="submit">Delete Post</button>
</form>
```

## TOPIC 10: COMPLETE RESTFUL CRUD (VS CODE STYLE)

### Full Production-Grade Express.js RESTful Architecture:

```
server.js - Full RESTful CRUD Server

const express = require('express');
const app = express();
const { v4: uuidv4 } = require('uuid');
const methodOverride = require('method-override');

app.use(express.urlencoded({ extended: true }));
app.use(methodOverride('_method'));
app.set('view engine', 'ejs');

let posts = [
  { id: uuidv4(), username: "shivam", content: "Mastering REST APIs!" },
  { id: uuidv4(), username: "rahul", content: "Express makes routing seamless!" }
];

// 1. INDEX: GET /posts (Read all)
app.get('/posts', (req, res) => {
  res.render('posts/index.ejs', { posts });
});

// 2. NEW: GET /posts/new (Render creation form)
app.get('/posts/new', (req, res) => {
  res.render('posts/new.ejs');
});

// 3. CREATE: POST /posts (Persist new resource)
app.post('/posts', (req, res) => {
  const { username, content } = req.body;
  posts.push({ id: uuidv4(), username, content });
  res.redirect('/posts'); // PRG Pattern
});

// 4. SHOW: GET /posts/:id (Read individual post)
app.get('/posts/:id', (req, res) => {
  const post = posts.find(p => p.id === req.params.id);
  res.render('posts/show.ejs', { post });
});

// 5. EDIT: GET /posts/:id/edit (Render edit form)
app.get('/posts/:id/edit', (req, res) => {
  const post = posts.find(p => p.id === req.params.id);
  res.render('posts/edit.ejs', { post });
});

// 6. UPDATE: PATCH /posts/:id (Modify post content)
app.patch('/posts/:id', (req, res) => {
  const post = posts.find(p => p.id === req.params.id);
  post.content = req.body.content;
  res.redirect('/posts');
});

// 7. DESTROY: DELETE /posts/:id (Remove post)
app.delete('/posts/:id', (req, res) => {
  posts = posts.filter(p => p.id !== req.params.id);
  res.redirect('/posts');
});

app.listen(8080, () => console.log("REST Server listening on port 8080"));
```